JOHN THE RIPPER

JOHN THE RIPPER

John the Ripper is a well-known, well-loved, and versatile hash-cracking tool. It combines a fast cracking speed with an extraordinary range of compatible hash types.

A hash is a way of taking a piece of data of any length and representing it in another fixed-length form. This process masks the original value of the data. The hash value is obtained by running the original data through a hashing algorithm. Many popular hashing algorithms exist, such as MD4, MD5, SHA1 and NTLM

If we take “polo”, a string of four characters, and run it through an MD5 hashing algorithm, we end up with an output of b53759f3ce692de7aff1b5779d3964da, a standard 32-character MD5 hash.

Hashing functions are designed as one-way functions. In other words, it is easy to calculate the hash value of a given input; however, it is a hard problem to find the original input given the hash value. In simple terms, a hard problem quickly becomes computationally infeasible in computer science. This computational problem has its roots in mathematics as P vs NP.

In computer science, P and NP are two classes of problems that help us understand the efficiency of algorithms:

P (Polynomial Time): Class P covers the problems whose solution can be found in polynomial time. Consider sorting a list in increasing order. The longer the list, the longer it would take to sort; however, the increase in time is not exponential.

NP (Non-deterministic Polynomial Time): Problems in the class NP are those for which a given solution can be checked quickly, even though finding the solution itself might be hard. In fact, we don’t know if there is a fast algorithm to find the solution in the first place.

the algorithm to hash the value will be “P” and can, therefore, be calculated reasonably. However, an “un-hashing” algorithm would be “NP” and intractable to solve, meaning that it cannot be computed in a reasonable time using standard computers.

JOHN PLAYS ITS ROLE

Even though the algorithm is not feasibly reversible, that doesn’t mean cracking the hashes is impossible. If you have the hashed version of a password, for example, and you know the hashing algorithm, you can use that hashing algorithm to hash a large number of words, called a dictionary. You can then compare these hashes to the one you’re trying to crack to see if they match. If they do, you know what word corresponds to that hash- you’ve cracked it!

This process is called a dictionary attack, and John the Ripper, or John as it’s commonly shortened, is a tool for conducting fast brute force attacks on various hash types.

the most popular extended version of John the Ripper, Jumbo John.

Kali Linux distributions, the /usr/share/wordlists directory contains a series of great wordlists.

RockYou

For all of the tasks in this room, we will use the infamous rockyou.txt wordlist, a very large common password wordlist obtained from a data breach on a website called rockyou.com in 2009.

If you are not using any of the above distributions, you can get the rockyou.txt wordlist from the SecLists repository under the /Passwords/Leaked-Databases subsection.

You may need to extract it from the .tar.gz format using tar xvzf rockyou.txt.tar.gz.

John Basic Syntax

The basic syntax of John the Ripper commands is as follows. We will cover the specific options and modifiers used as we use them.

john [options] [file path]

john: Invokes the John the Ripper program
[options]: Specifies the options you want to use
[file path]: The file containing the hash you’re trying to crack; if it’s in the same directory, you won’t need to name a path, just the file.

Automatic Cracking

John has built-in features to detect what type of hash it’s being given and to select appropriate rules and formats to crack it for you; this isn’t always the best idea as it can be unreliable, but if you can’t identify what hash type you’re working with and want to try cracking it, it can be a good option! To do this, we use the following syntax:

john --wordlist=[path to wordlist] [path to file]

- -wordlist=: Specifies using wordlist mode, reading from the file that you supply in the provided path

[path to wordlist]: The path to the wordlist you’re using, as described in the previous task

Example Usage:

john --wordlist=/usr/share/wordlists/rockyou.txt hash_to_crack.txt

We can use other tools to identify the hash and then set John to a specific format. There are multiple ways to do this, such as using an online hash identifier like this site. I like to use a tool called hash-identifier, a Python tool that is super easy to use and will tell you what different types of hashes the one you enter is likely to be, giving you more options if the first one fails.

To use hash-identifier, you can use wget or curl to download the Python file hash-id.py from its GitLab page. Then, launch it with python3 hash-id.py and enter the hash you’re trying to identify. It will give you a list of the most probable formats. These two steps are shown in the terminal below.

wget https://gitlab.com/kalilinux/packages/hash-identifier/-/raw/kali/master/hash-id.py
$ python3 hash-id.py

john --format=[format] --wordlist=[path to wordlist] [path to file]

- -format=: This is the flag to tell John that you’re giving it a hash of a specific format and to use the following format to crack it
  [format]: The format that the hash is in

john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash_to_crack.txt

A Note on Formats:

When you tell John to use formats, if you’re dealing with a standard hash type, e.g. md5 as in the example above, you have to prefix it with raw- to tell John you’re just dealing with a standard hash type, though this doesn’t always apply.
To check if you need to add the prefix or not, you can list all of John’s formats using john --list=formats and either check manually or grep for your hash type using something like john --list=formats | grep -iF "md5".

COMMAND USING JOHN THE RIPPER (by me)

USING MD5

```
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task04$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash1.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
biscuit          (?)

1g 0:00:00:00 DONE (2025-11-03 08:17) 50.00g/s 134400p/s 134400c/s 134400C/s skyblue..nugget
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

SECOND ONE USING SHA1

```
ser@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task04$ john --format=raw-sha1 --wordlist=/usr/share/wordlists/rockyou.txt hash2.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 256/256 AVX2 8x])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
kangeroo        (?)

1g 0:00:00:00 DONE (2025-11-03 08:19) 33.33g/s 3904Kp/s 3904Kc/s 3904KC/s karakara..kalinda
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.
```

```
ls
hash-id.py  hash1.txt  hash2.txt  hash3.txt  hash4.txt
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task04$ cat hash3.txt
D7F4D3CCEE7ACD3DD7FAD3AC2BE2AAE9C44F4E9B7FB802D73136D4C53920140A

user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task04$ type hash3.txt
-bash: type: hash3.txt: not found

user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task04$ python3 hash-id.py
/home/user/John-the-Ripper-The-Basics/Task04/hash-id.py:13: SyntaxWarning: invalid escape sequence '\ '
logo='''   #############################################################################
#############################################################################

__ __ __ _____ _____

/\ \/\ \ /\ \ /\__ _\ /\ _ `\

\ \ \\ \ \__ ____ \ \ \__ \/_/\ \/ \ \ \/\ \

\ \ _ \ /'__ \\ / ,__\\ \\ \\ _ \ \ \ \ \ \ \ \

\ \ \ \ \/\ \\ \ \ /\\, `\ \ \ \ \ \ \_\ \ \ \ \ \_\ \

\ \\ \\ \___ \\/\/ \ \ \\ \\ /\_____\ \ \___/

\/ \\/\/\\/\\/ \\/\/ \\/\/ \\__/ \\/ v1.2

By Zion3R

www.Blackploit.com

Root@Blackploit.com

#############################################################################
HASH: D7F4D3CCEE7ACD3DD7FAD3AC2BE2AAE9C44F4E9B7FB802D73136D4C53920140A

Possible Hashs:
[+] SHA-256
[+] Haval-256

Least Possible Hashs:
[+] GOST R 34.11-94
[+] RipeMD-256
[+] SNEFRU-256
[+] SHA-256(HMAC)
[+] Haval-256(HMAC)
[+] RipeMD-256(HMAC)
[+] SNEFRU-256(HMAC)
[+] SHA-256(md5($pass))
[+] SHA-256(sha1($pass))

john --format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt hash3.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
microphone      (?)

1g 0:00:00:00 DONE (2025-11-03 08:23) 33.33g/s 3276Kp/s 3276Kc/s 3276KC/s rozalia..Dominic1
Use the "--show --format=Raw-SHA256" options to display all of the cracked passwords reliably
Session completed.
```

```
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task04$ cat hash4.txt
c5a60cc6bbba781c601c5402755ae1044bbf45b78d1183cbf2ca1c865b6c792cf3c6b87791344986c8a832a0f9ca8d0b4afd3d9421a149d57075e1b4e93f90bf
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task04$ python3 hash-id.py
/home/user/John-the-Ripper-The-Basics/Task04/hash-id.py:13: SyntaxWarning: invalid escape sequence '\ '
logo='''   #############################################################################
#############################################################################
```

```
__ __ __ _____ ____
/\ \/\ \ /\ \ /\__ _\ /\ _ `\
\ \ \\ \ \ __ ____ \ \ \ \__ \/_/\ \/ \ \ \/\ \
\ \ _ \ /'__`\\ \ / ,__\\ \\ \\ _ \ \ \ \ \ \ \ \ \
\ \ \ \ \/\ \\ \ \ /\\, `\ \ \ \ \ \ \ \_\ \ \ \ \ \_\ \
\ \\ \ \\ \___ \ \/\/ \ \ \\ \\ /\_____\ \ \ \___/
\/ \/\/\/\/\/ \/\/ \/\/ \/\/ \/__/ \/\/ v1.2
```

By Zion3R

www.Blackploit.com

Root@Blackploit.com

##

HASH: c5a60cc6bbba781c601c5402755ae1044bbf45b78d1183cbf2ca1c865b6c792cf3c6b87791344986c8a832a0f9ca8d0b4afd3d9421a149d57075e1b4e93f90bf

Possible Hashs:
[+] SHA-512
[+] Whirlpool

Least Possible Hashs:
[+] SHA-512(HMAC)
[+] Whirlpool(HMAC)

```
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task04$ john --format=whirlpool --wordlist=/usr/share/wordlists/rockyou.txt hash4.txt
Using default input encoding: UTF-8
Loaded 1 password hash (whirlpool [WHIRLPOOL 32/64])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
colossal         (?)

1g 0:00:00:00 DONE (2025-11-03 08:27) 2.326g/s 1581Kp/s 1581Kc/s 1581KC/s cooldog12..chata1994
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

---

tool used - john the ripper , hash-id.py

Cracking Windows Authentication Hashes

Authentication hashes are the hashed versions of passwords stored by operating systems; it is sometimes possible to crack them using our brute-force methods. To get your hands on these hashes, you must often already be a privileged user, so we will explain some of the hashes we plan on cracking as we attempt them.

NTHash / NTLM

NThash is the hash format modern Windows operating system machines use to store user and service passwords.
It's also commonly referred to as NTLM, which references the previous version of Windows format for hashing passwords known as LM, thus NT/LM.

A bit of history: the NT designation for Windows products originally meant New Technology. It was used starting with Windows NT to denote products not built from the MS-DOS Operating System.
Eventually, the "NT" line became the standard Operating System type to be released by Microsoft, and the name was dropped, but it still lives on in the names of some Microsoft technologies.

In Windows, SAM (Security Account Manager) is used to store user account information, including usernames and hashed passwords. You can acquire NTHash/NTLM hashes by dumping the SAM database on a Windows machine, using a tool like Mimikatz, or using the Active Directory database: NTDS.dit. You may not have to crack the hash to continue privilege escalation, as you can often conduct a "pass the hash" attack instead, but sometimes, hash cracking is a viable option if there is a weak password policy.

ATTACK (NTLM USED AS NT IN FORMAT)

```
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task05$ john -format=nt --wordlist=/usr/share/wordlists/rockyou.txt ntlm.txt
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 27 rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
mushroom         (?)

1g 0:00:00:00 DONE (2025-11-03 08:44) 50.00g/s 153600p/s 153600c/s 153600C/s skater1..dangerous
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.
```

Cracking Hashes from /etc/shadow

The /etc/shadow file is the file on Linux machines where password hashes are stored.
It also stores other information, such as the date of last password change and password expiration information.
It contains one entry per line for each user or user account of the system.
This file is usually only accessible by the root user, so you must have sufficient privileges to access the hashes.
However, if you do, there is a chance that you will be able to crack some of the hashes.

Unshadowing

John can be very particular about the formats it needs data in to be able to work with it; for this reason, to crack /etc/shadow passwords, you must combine it with the /etc/passwd file for John to understand the data it's being given. To do this, we use a tool built into the John suite of tools called unshadow.

- The basic syntax of unshadow is as follows:

unshadow [path to passwd] [path to shadow]

unshadow: Invokes the unshadow tool
[path to passwd]: The file that contains the copy of the /etc/passwd file you've taken from the target machine
[path to shadow]: The file that contains the copy of the /etc/shadow file you've taken from the target machine

Example Usage:
unshadow local_passwd local_shadow > unshadowed.txt

Note on the files

When using unshadow, you can either use the entire /etc/passwd and /etc/shadow files, assuming you have them available, or you can use the relevant line from each, for example:

FILE 1 - local_passwd

Contains the /etc/passwd line for the root user:

root:x:0:0::/root:/bin/bash

FILE 2 - local_shadow

Contains the /etc/shadow line for the root user: root:$6$2nwjN454g.dv4HN/$m9Z/r2xVfweYVkrr.v5Ft8Ws3/YYksfNwq96UL1FX0OJjY1L6I.DS3KEVsZ9rOVLB/I

Cracking

We can then feed the output from unshadow, in our example use case called unshadowed.txt, directly into John. We should not need to specify a mode here as we have made the input specifically for John; however, in some cases, you will need to specify the format as we have done previously using: --format=sha512crypt

john --wordlist=/usr/share/wordlists/rockyou.txt --format=sha512crypt unshadowed.txt

---

SIMULATING

```
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task06$ unshadow local_passwd local_shadow > unshadowed.txt

user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task06$ john --wordlist=unshadowed.txt  etc_hashes.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cracked 1 password hash (is in /home/user/src/john/run/john.pot), use "--show"
No password hashes left to crack (see FAQ)
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task06$ cd /home/user/src/john/run/

user@ip-10-201-106-58:~/src/john/run$ ls
1password2john.py          genincstats.rb          pass_gen.pl
7z2john.pl                 genmkvpwd               password.lst
DPAPImk2john.py            gpg2john                pcap2john.py
SIPdump                    hccap2john              pdf2john.pladxcsouf2john.py          hccapx2john.py          pdf2john.pyaem2john.py                hextoraw.pl
pem2john.pyaix2john.pl     htdigest2john.py        pfx2john.pyaix2john.py               hybrid.conf             pgpdisk2john.py
alnum.chr                  ibmiscanner2john.py     pgpsda2john.py
alnumspace.chr             ikescan2john.py         pgpwde2john.py
alpha.chr                  ios7tojohn.pl           pkcs12kdf.pyandotp2john.py           itunes_backup2john.pl   potcheck.plandroidbackup2john.py      iwork2john.py
prosody2john.pyandroidfde2john.py  john             protobuf
ansible2john.py            john.bash_completion    ps_token2john.py
apex2john.py               john.conf               pse2john.pyapop2john.py              john.log                putty2john
applenotes2john.py         john.pot                pwsafe2john.pyaruba2john.py          john.zsh_completion     racf2john
ascii.chr                  jtr_rulez.pm            radius2john.platmail2john.pl         jtrconf.pm              radius2john.pyaxcrypt2john.py         kdcdump2john.py
rar2john
base64conv                 keepass2john            raw2dyna
benchmark-unify            keplr2john.py           regex_alphabets.conf
bestcrypt2john.py          keychain2john.py        relbench
bestcryptve2john.py        keyring2john.py         repeats16.conf
bip-0039                   keystore2john.py        repeats32.conf
bitcoin2john.py            kirbi2john.py           restic2john.py
bitlocker2john             known_hosts2john.py     rexgen2rules.plbitshares2john.py     korelogic.conf          rules
bitwarden2john.py          krb2john.py             rules-by-rate.conf
bks2john.py                kwallet2john.py         rules-by-score.conf
blockchain2john.py         lanman.chr              rulestack.pl
calc_stat                  lastpass2john.py        sap2john.plcardano2john.py           latin1.chr              sense2john.pyccache2john.py           ldif2john.pl            sha-
dump.pl
ccl_chrome_indexeddb       leet.pl                 sha-test.plcisco2john.pl             lib                     signal2john.pycodepage.pl             libreoffice2john.py
sipdump2john.pycoinomi2john.py     lion2john-alt.pl        ssh2john.py
cprepair                   lion2john.pl            sspr2john.pycracf2john.py            lm_ascii.chr            staroffice2john.pydashlane2john.py    lotus2john.py           stats
deepsound2john.py          lower.chr               strip2john.py
dictionary.rfc2865         lowernum.chr            telegram2john.py
digits.chr                 lowerspace.chr          test_tezos2john.py
diskcryptor2john.py        luks2john.py            tezos2john.py
dmg2john                   mac2john-alt.py         tgtsnarf
dmg2john.py                mac2john.py             truecrypt2john.py
dns                        mailer                  uaf2john
dumb16.conf                makechr                 unafs
dumb32.conf                mcafee_epo2john.py      undrop
dynamic.conf               mkvcalcproba            unique
dynamic_disabled.conf      monero2john.py          unisubst.conf
dynamic_flat_sse_formats.conf  money2john.py       unrule.pl
eapmd5tojohn               mongodb2john.js         unshadow
ecryptfs2john.py           mosquitto2john.py       upper.chr
ejabberd2john.py           mozilla2john.py         uppernum.chr
electrum2john.py           multibit2john.py        utf8.chr
encdatavault2john.py       neo2john.py             vdi2john.plencfs2john.py             netntlm.pl              vmx2john.pyenpass2john.py             netscreen.py
vncpcap2john
enpass5tojohn.py           network2john.lua        wpapcap2john
ethereum2john.py           office2john.py          zed2john.pyfilezilla2john.py         openbsd_softraid2john.py  zip2john
fuzz.dic                   opencl                  ztex
fuzz_option.pl             openssl2john.pygeli2john.py          padlock2john.py
```

```
user@ip-10-201-106-58:~/src/john/run$ cat john.pot
$dynamic_0$2e728dd31fb5949bc39cac5a9f066498:biscuit
$dynamic_26$1a732667f3917c0f4aa98bb13011b9090c6f8065:kangeroo
```

$SHA256$d7f4d3ccee7acd3dd7fad3ac2be2aae9c44f4e9b7fb802d73136d4c53920140a:microphone
$whirlpool$C5A60CC6BBBA781C601C5402755AE1044BBF45B78D1183CBF2CA1C865B6C792CF3C6B87791344986C8A832A0F9CA8D0B4AFD3D9421A149D57075E1B4E9
NT5460c85bd858a11475115d2dd3a82333:mushroom
6Ha.d5nGupBm29pYr$yugXSk24ZljLTAZZagtGwpSQhb3F2DOJtnHrvk7HI2ma4GsuioHp8sm3LJiRJpKfIf7IZQ29qgtH17Q/JDpYM/:(1234) THIS IS FLAG !! Root password found

---

Single Crack Mode

So far, we've been using John's wordlist mode to brute-force simple and not-so-simple hashes. But John also has another mode, called the Single Crack mode. In this mode, John uses only the information provided in the username to try and work out possible passwords heuristically by slightly changing the letters and numbers contained within the username.

Word Mangling

The best way to explain Single Crack mode and word mangling is to go through an example:

Consider the username "Markus".

Some possible passwords could be:

Markus1, Markus2, Markus3 (etc.)
MArkus, MARkus, MARKus (etc.)
Markus!, Markus$, Markus* (etc.)

This technique is called word mangling. John is building its dictionary based on the information it has been fed and uses a set of rules called "mangling rules," which define how it can mutate the word it started with to generate a wordlist based on relevant factors for the target you're trying to crack. This exploits how poor passwords can be based on information about the username or the service they're logging into.

GECOS

John's implementation of word mangling also features compatibility with the GECOS field of the UNIX operating system, as well as other UNIX-like operating systems such as Linux. GECOS stands for General Electric Comprehensive Operating System. In the last task, we looked at the entries for both /etc/shadow and /etc/passwd. Looking closely, you will notice that the fields are separated by a colon :. The fifth field in the user account record is the GECOS field. It stores general information about the user, such as the user's full name, office number, and telephone number, among other things. John can take information stored in those records, such as full name and home directory name, to add to the wordlist it generates when cracking /etc/shadow hashes with single crack mode.

Using Single Crack Mode

To use single crack mode, we use roughly the same syntax that we've used so far; for example, if we wanted to crack the password of the user named "Mike", using the single mode, we'd use:

john --single --format=[format] [path to file]

- -single: This flag lets John know you want to use the single hash-cracking mode
  --format=[format]: As always, it is vital to identify the proper format.

Example Usage:
john --single --format=raw-sha256 hashes.txt

A Note on File Formats in Single Crack Mode:

If you're cracking hashes in single crack mode, you need to change the file format that you're feeding John for it to understand what data to create a wordlist from.
You do this by prepending the hash with the username that the hash belongs to, so according to the above example, we would change the file hashes.txt

From 1efee03cdcb96d90ad48ccc7b8666033

To mike:1efee03cdcb96d90ad48ccc7b8666033

SIMULATION

nano hash07.txt
Joker:7bf6d9bb82bed1302f331fc6b816aada

john --single --format=raw-md5 hash07.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
Warning: Only 18 candidates buffered for the current salt, minimum 24 needed for performance.
Jok3r (Joker)

1g 0:00:00:00 DONE (2025-11-03 09:23) 50.00g/s 9750p/s 9750c/s 9750C/s Joker!!..J0ker
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

Cracking Password Protected Zip Files

Zip2John

Similarly to the unshadow tool we used previously, we will use the zip2john tool to convert the Zip file into a hash format that John can understand and hopefully crack. The primary usage is like this:

zip2john [options] [zip file] > [output file]

[options]: Allows you to pass specific checksum options to zip2john; this shouldn't often be necessary
[zip file]: The path to the Zip file you wish to get the hash of

> : This redirects the output from this command to another file
> [output file]: This is the file that will store the output
> Example Usage

zip2john zipfile.zip > zip_hash.txt

Cracking

We're then able to take the file we output from zip2john in our example use case, zip_hash.txt, and, as we did with unshadow, feed it directly into John as we have made the input specifically for it.

john --wordlist=/usr/share/wordlists/rockyou.txt zip_hash.txt

---

SIMULATION

user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task09$ ls
secure.zip
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task09$ zip2john secure.zip >zip_hash.txt
ver 1.0 efh 5455 efh 7875 secure.zip/zippy/flag.txt PKZIP Encr: 2b chk, TS_chk, cmplen=38, decmplen=26, crc=849AB5A6 ts=B689 cs=b689 type=0

```
john --wordlist=/usr/share/wordlists/rockyou.txt zip_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 2 OpenMP threads
Note: Passwords longer than 21 [worst case UTF-8] to 63 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
pass123          (secure.zip/zippy/flag.txt)

1g 0:00:00:00 DONE (2025-11-03 09:29) 50.00g/s 409600p/s 409600c/s 409600C/s newzealand..total90
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

```
ls
secure.zip  zip_hash.txt
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task09$ unzip secure.zip
Archive:  secure.zip
[secure.zip] zippy/flag.txt password:
extracting: zippy/flag.txt

user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task09$ ls
secure.zip  zip_hash.txt  zippy
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task09$ cd zippy
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task09/zippy$ ls
flag.txt
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task09/zippy$ cat flag.txt
THM{w3ll_d0n3_h4sh_r0y4l}
```

CUSTOM RULES

What are Custom Rules?
As we explored what John can do in Single Crack Mode, you may have some ideas about some good mangling patterns or what patterns your passwords often use that could be replicated with a particular mangling pattern. The good news is that you can define your rules, which John will use to create passwords dynamically. The ability to define such rules is beneficial when you know more information about the password structure of whatever your target is.

Common Custom Rules
Many organisations will require a certain level of password complexity to try and combat dictionary attacks. In other words, when creating a new account or changing your password, if you attempt a password like polopassword, it will most likely not work. The reason would be the enforced password complexity. As a result, you may receive a prompt telling you that passwords have to contain at least one character from each of the following:

Lowercase letter
Uppercase letter
Number
Symbol
Password complexity is good! However, we can exploit the fact that most users will be predictable in the location of these symbols. For the above criteria, many users will use something like the following:

Polopassword1!

Consider the password with a capital letter first and a number followed by a symbol at the end. This familiar pattern of the password, appended and prepended by modifiers (such as capital letters or symbols), is a memorable pattern that people use and reuse when creating passwords. This pattern can let us exploit password complexity predictability.

Now, this does meet the password complexity requirements; however, as attackers, we can exploit the fact that we know the likely position of these added elements to create dynamic passwords from our wordlists.

How to create Custom Rules
Custom rules are defined in the john.conf file. This file can be found in /opt/john/john.conf on the TryHackMe Attackbox. It is usually located in /etc/john/john.conf if you have installed John using a package manager or built from source with make.

Let's go over the syntax of these custom rules, using the example above as our target pattern. Note that you can define a massive level of granular control in these rules. I suggest looking at the wiki here to get a full view of the modifiers you can use and more examples of rule implementation.

The first line:

[List.Rules:THMRules] is used to define the name of your rule; this is what you will use to call your custom rule a John argument.

We then use a regex style pattern match to define where the word will be modified; again, we will only cover the primary and most common modifiers here:

Az: Takes the word and appends it with the characters you define
A0: Takes the word and prepends it with the characters you define
c: Capitalises the character positionally
These can be used in combination to define where and what in the word you want to modify.

Lastly, we must define what characters should be appended, prepended or otherwise included. We do this by adding character sets in square brackets [ ] where they should be used. These follow the modifier patterns inside double quotes " ". Here are some common examples:

[0-9]: Will include numbers 0-9
[0]: Will include only the number 0
[A-z]: Will include both upper and lowercase
[A-Z]: Will include only uppercase letters
[a-z]: Will include only lowercase letters
Please note that:

[a]: Will include only a
[!£$%@]: Will include the symbols !, £, $, %, and @
Putting this all together, to generate a wordlist from the rules that would match the example password Polopassword1! (assuming the word polopassword was in our wordlist), we would create a rule entry that looks like this:

[List.Rules:PoloPassword]

cAz"[0-9] [!£$%@]"

Utilises the following:

c: Capitalises the first letter
Az: Appends to the end of the word
[0-9]: A number in the range 0-9
[!£$%@]: The password is followed by one of these symbols
Using Custom Rules
We could then call this custom rule a John argument using the --rule=PoloPassword flag.

As a full command: john --wordlist=[path to wordlist] --rule=PoloPassword [path to file]

As a note, I find it helpful to talk out the patterns if you're writing a rule; as shown above, the same applies to writing RegEx patterns.

Jumbo John already has an extensive list of custom rules containing modifiers for use in almost all cases. If you get stuck, try looking at those rules [around line 678] if your syntax isn't working correctly.

Cracking a Password-Protected RAR Archive (SAME AS ZIP2JOHN)

We can use a similar process to the one we used in the last task to obtain the password for RAR archives. If you aren't familiar, RAR archives are compressed files created by the WinRAR archive manager. Like Zip files, they compress folders and files.

Rar2John

Almost identical to the zip2john tool, we will use the rar2john tool to convert the RAR file into a hash format that John can understand. The basic syntax is as follows:

rar2john [rar file] > [output file]

rar2john: Invokes the rar2john tool
[rar file]: The path to the RAR file you wish to get the hash of

> : This redirects the output of this command to another file
> [output file]: This is the file that will store the output from the command
> Example Usage

/opt/john/rar2john rarfile.rar > rar_hash.txt

Cracking

Once again, we can take the file we output from rar2john in our example use case, rar_hash.txt, and feed it directly into John as we did with zip2john.

john --wordlist=/usr/share/wordlists/rockyou.txt rar_hash.txt

---

SIMULATING

```
cd Task10
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task10$ ls
secure.rar
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task10$ rar2john secure.rar >rar_hash.txt
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task10$ john --wordlist=/usr/share/wordlists/rockyou.txt rar_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (RAR5 [PBKDF2-SHA256 256/256 AVX2 8x])
Cost 1 (iteration count) is 32768 for all loaded hashes
Will run 2 OpenMP threads
Note: Passwords longer than 10 [worst case UTF-8] to 32 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
password         (secure.rar)

1g 0:00:00:00 DONE (2025-11-03 10:05) 1.667g/s 106.7p/s 106.7c/s 106.7C/s 123456..charlie
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Cracking SSH Key Passwords

Okay, okay, I hear you. There are no more file archives! Fine! Let's explore one more use of John that comes up semi-frequently in CTF challenges—using John to crack the SSH private key password of id_rsa files. Unless configured otherwise, you authenticate your SSH login using a password. However, you can configure key-based authentication, which lets you use your private key, id_rsa, as an authentication key to log in to a remote machine over SSH. However, doing so will often require a password to access the private key; here, we will be using John to crack this password to allow authentication over SSH using the key.

SSH2John

Who could have guessed it, another conversion tool? Well, that's what working with John is all about. As the name suggests, ssh2john converts the id_rsa private key, which is used to log in to the SSH session, into a hash format that John can work with. Jokes aside, it's another beautiful example of John's versatility. The syntax is about what you'd expect. Note that if you don't have ssh2john installed, you can use ssh2john.py, located in the /opt/john/ssh2john.py. If you're doing this on the AttackBox, replace the ssh2john command with python3 /opt/john/ssh2john.py or on Kali, python /usr/share/john/ssh2john.py.

ssh2john [id_rsa private key file] > [output file]

ssh2john: Invokes the ssh2john tool
[id_rsa private key file]: The path to the id_rsa file you wish to get the hash of

> : This is the output director. We're using it to redirect the output from this command to another file.
> [output file]: This is the file that will store the output from

Example Usage

/opt/john/ssh2john.py id_rsa > id_rsa_hash.txt

Cracking

For the final time, we're feeding the file we output from ssh2john, which in our example use case is called id_rsa_hash.txt and, as we did with rar2john, we can use this seamlessly with John:

john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa_hash.txt

---

SIMULATION OF SSH2JOHN

```
cd Task11
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task11$ ls
id_rsa  id_rsa_hash.txt
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task11$ /opt/john/ssh2john.py id_rsa > id_rsa_hash.txt
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task11$ ls
id_rsa  id_rsa_hash.txt
user@ip-10-201-106-58:~/John-the-Ripper-The-Basics/Task11$ john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 2 OpenMP threads
Note: Passwords longer than 10 [worst case UTF-8] to 32 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
mango            (id_rsa)

1g 0:00:00:00 DONE (2025-11-03 10:21) 50.00g/s 214400p/s 214400c/s 214400C/s praise..mango
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```